

Modular Shader & Mathematical Graphics Toolbox

A production-ready reference manual and copy-paste toolbox of GLSL primitives, spatial transformations, procedural noise algorithms, Signed Distance Fields (SDFs), colorimetric conversions, and analytical lighting models.

Table of Contents

1. [Coordinate Spaces & Aspect Correction](#)
2. [Geometric Transformations & Linear Algebra](#)
3. [Shaping Functions & Analytic Curves](#)
4. [Procedural Coherent Noise & Field Generators](#)
5. [Domain Warping & Fractal Synthesis](#)
6. [2D Signed Distance Fields \(SDFs\) & Operators](#)
7. [Procedural Tiling, Lattices & Symmetry](#)
8. [Color Science & Spectral Synthesis](#)
9. [Analytical Lighting, Normals & Shading](#)

1. Coordinate Spaces & Aspect Correction

Fragment shaders receive raw screen coordinates in pixels via `gl_FragCoord.xy`. To achieve resolution-independent layouts, convert them to normalized, aspect-corrected coordinate frames.

```
// Normalized Device Coordinates (NDC) mapped to [0.0, 1.0]
vec2 getUV(in vec2 fragCoord, in vec2 u_resolution) {
    return fragCoord.xy / u_resolution.xy;
}

// Centered coordinates: Origin (0,0) at viewport center, y in [-1.0, 1.0]
// Corrects for non-square pixel aspect ratios without stretching
vec2 getAspectCorrectedUV(in vec2 fragCoord, in vec2 u_resolution) {
    vec2 uv = (2.0 * fragCoord - u_resolution.xy) / u_resolution.y;
    return uv;
}

// Screen-to-UV remapping with zoom and pan
vec2 getTransformedUV(in vec2 fragCoord, in vec2 u_resolution, in vec2
pan, in float zoom) {
    vec2 uv = (2.0 * fragCoord - u_resolution.xy) / u_resolution.y;
    return (uv - pan) / zoom;
}
```

2. Geometric Transformations & Linear Algebra

2.1 2D Rotation & Skew

Rotation preserves Euclidean distance:

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

```
// 2D Rotation matrix around origin
mat2 rotate2D(in float angle) {
    float c = cos(angle);
    float s = sin(angle);
    return mat2(c, -s, s, c);
}

// Skew / Shear matrix
vec2 skew2D(in vec2 p, in vec2 skewAmount) {
    return vec2(
        p.x + p.y * tan(skewAmount.x),
        p.y + p.x * tan(skewAmount.y)
    );
}
```

2.2 3D Euler & Rodrigues Rotations

```
// 3D Rotation along principal axes
mat3 rotateX(in float theta) {
    float c = cos(theta), s = sin(theta);
    return mat3(1.0, 0.0, 0.0, 0.0, c, -s, 0.0, s, c);
}

mat3 rotateY(in float theta) {
    float c = cos(theta), s = sin(theta);
    return mat3(c, 0.0, s, 0.0, 1.0, 0.0, -s, 0.0, c);
}

mat3 rotateZ(in float theta) {
    float c = cos(theta), s = sin(theta);
    return mat3(c, -s, 0.0, s, c, 0.0, 0.0, 0.0, 1.0);
}

// Arbitrary axis rotation via Rodrigues' formula
mat3 rotateAxis(in vec3 axis, in float angle) {
    vec3 a = normalize(axis);
    float s = sin(angle);
```

```

float c = cos(angle);
float oc = 1.0 - c;

return mat3(
    oc * a.x * a.x + c,          oc * a.x * a.y - a.z * s,  oc * a.z
* a.x + a.y * s,
    oc * a.x * a.y + a.z * s,  oc * a.y * a.y + c,          oc * a.y
* a.z - a.x * s,
    oc * a.z * a.x - a.y * s,  oc * a.y * a.z + a.x * s,  oc * a.z
* a.z + c
);
}

```

2.3 Non-Linear Polar & Conformal Transforms

```

// Cartesian (x, y) to Polar (r, theta)
vec2 toPolar(in vec2 p) {
    float r = length(p);
    float theta = atan(p.y, p.x); // [-PI, PI]
    return vec2(r, theta);
}

// Polar to Cartesian
vec2 toCartesian(in vec2 polar) {
    return vec2(polar.x * cos(polar.y), polar.x * sin(polar.y));
}

// Log-Spherical / Complex Exponential Spiral Map
vec2 complexLogMap(in vec2 p) {
    float r = length(p);
    float theta = atan(p.y, p.x);
    return vec2(log(r), theta);
}

// Swirl / Vortex Deformation
vec2 swirl(in vec2 p, in float strength, in float radius) {
    float dist = length(p);
    if (dist < radius) {
        float percent = (radius - dist) / radius;
        float theta = percent * percent * strength;
        return rotate2D(theta) * p;
    }
    return p;
}

```

3. Shaping Functions & Analytic Curves

3.1 Easing & Interpolation Formulas

```
// Hermite Smoothstep variations
float smoothStepHermite(in float edge0, in float edge1, in float x) {
    float t = clamp((x - edge0) / (edge1 - edge0), 0.0, 1.0);
    return t * t * (3.0 - 2.0 * t);
}

// Ken Perlin's Quintic Smootherstep (zero 1st & 2nd derivatives at
endpoints)
float smootherstep(in float edge0, in float edge1, in float x) {
    float t = clamp((x - edge0) / (edge1 - edge0), 0.0, 1.0);
    return t * t * t * (t * (t * 6.0 - 15.0) + 10.0);
}

// Normalized Gain & Bias (Schlick)
float bias(in float b, in float x) {
    return x / ((1.0 / b - 2.0) * (1.0 - x) + 1.0);
}

float gain(in float g, in float x) {
    if (x < 0.5) {
        return bias(1.0 - g, 2.0 * x) * 0.5;
    } else {
        return 1.0 - bias(1.0 - g, 2.0 - 2.0 * x) * 0.5;
    }
}
```

3.2 Impulse & Attenuation Curves

```
// Inigo Quilez's Impulse: peaks at x = 1/k with peak value = 1.0
float impulse(in float k, in float x) {
    float h = k * x;
    return h * exp(1.0 - h);
}

// Sustained Exponential Impulse: control attack and decay separately
float expSustainedImpulse(in float x, in float f, in float k) {
    float s = max(x - f, 0.0);
    return min(x * x / (f * f), 1.0 + (2.0 / f) * s * exp(-k * s));
}

// Parabola centered at 0.5 with width mapped to [0.0, 1.0]
float parabola(in float x, in float k) {
    return x * x * k + 0.5 * (1.0 - k);
}
```

```

    return pow(4.0 * x * (1.0 - x), k);
}

```

4. Procedural Coherent Noise & Field Generators

GPU-friendly pseudo-random number generators (PRNG) compute hashes on lattice points without texture lookups.

4.1 Dimensionless Hash Primitives

```

// Deterministic 1D hash
float hash11(in float p) {
    p = fract(p * 0.1031);
    p *= p + 33.33;
    p *= p + p;
    return fract(p);
}

// 2D -> 1D Hash
float hash21(in vec2 p) {
    vec3 p3 = fract(vec3(p.xy) * 0.1031);
    p3 += dot(p3, p3.yzx + 33.33);
    return fract((p3.x + p3.y) * p3.z);
}

// 2D -> 2D Gradient vector hash
vec2 hash22(in vec2 p) {
    vec3 p3 = fract(vec3(p.xy) * vec3(0.1031, 0.1030, 0.0973));
    p3 += dot(p3, p3.yzx + 33.33);
    return fract((p3.xx + p3.yz) * p3.zy);
}

```

4.2 Bilinear Value Noise

```

float valueNoise(in vec2 p) {
    vec2 i = floor(p);
    vec2 f = fract(p);

    // Quintic interpolation
    vec2 u = f * f * f * (f * (f * 6.0 - 15.0) + 10.0);

    float a = hash21(i + vec2(0.0, 0.0));
    float b = hash21(i + vec2(1.0, 0.0));
    float c = hash21(i + vec2(0.0, 1.0));
    float d = hash21(i + vec2(1.0, 1.0));
}

```

```

    return mix(mix(a, b, u.x), mix(c, d, u.x), u.y);
}

```

4.3 Classic 2D Gradient Noise (Perlin)

```

vec2 getGradient(in vec2 i) {
    // Generate normalized gradient direction
    float a = hash21(i) * 6.28318530718;
    return vec2(cos(a), sin(a));
}

float perlinNoise(in vec2 p) {
    vec2 i = floor(p);
    vec2 f = fract(p);

    vec2 u = f * f * f * (f * (f * 6.0 - 15.0) + 10.0);

    vec2 g00 = getGradient(i + vec2(0.0, 0.0));
    vec2 g10 = getGradient(i + vec2(1.0, 0.0));
    vec2 g01 = getGradient(i + vec2(0.0, 1.0));
    vec2 g11 = getGradient(i + vec2(1.0, 1.0));

    float d00 = dot(g00, f - vec2(0.0, 0.0));
    float d10 = dot(g10, f - vec2(1.0, 0.0));
    float d01 = dot(g01, f - vec2(0.0, 1.0));
    float d11 = dot(g11, f - vec2(1.0, 1.0));

    return mix(mix(d00, d10, u.x), mix(d01, d11, u.x), u.y);
}

```

4.4 2D Simplex Noise

Evaluated on a skewed triangular grid to avoid directional axis bias.

```

vec3 permute(in vec3 x) { return mod(((x * 34.0) + 1.0) * x, 289.0); }

float simplexNoise(in vec2 v) {
    const vec4 C = vec4(
        0.211324865405187, // (3.0 - sqrt(3.0)) / 6.0
        0.366025403784439, // 0.5 * (sqrt(3.0) - 1.0)
        -0.577350269189626, // -1.0 + 2.0 * C.x
        0.024390243902439 // 1.0 / 41.0
    );

    vec2 i = floor(v + dot(v, C.yy));
    vec2 x0 = v - i + dot(i, C.xx);

```

```

vec2 i1 = (x0.x > x0.y) ? vec2(1.0, 0.0) : vec2(0.0, 1.0);
vec4 x12 = x0.xyxy + C.xxzz;
x12.xy -= i1;

i = mod(i, 289.0);
vec3 p = permute(permute(i.y + vec3(0.0, i1.y, 1.0)) + i.x +
vec3(0.0, i1.x, 1.0));

vec3 m = max(0.5 - vec3(dot(x0, x0), dot(x12.xy, x12.xy),
dot(x12.zw, x12.zw)), 0.0);
m = m * m;
m = m * m;

vec3 x = 2.0 * fract(p * C.www) - 1.0;
vec3 h = abs(x) - 0.5;
vec3 ox = floor(x + 0.5);
vec3 a0 = x - ox;

m *= 1.79284291400159 - 0.85373472095314 * (a0 * a0 + h * h);

vec3 g;
g.x = a0.x * x0.x + h.x * x0.y;
g.yz = a0.yz * x12.xz + h.yz * x12.yw;
return 130.0 * dot(m, g);
}

```

4.5 Cellular / Worley Noise

```

// Computes closest distance (F1) and second closest distance (F2)
vec2 cellularNoise(in vec2 p) {
    vec2 n = floor(p);
    vec2 f = fract(p);

    float f1 = 8.0;
    float f2 = 8.0;

    for (int j = -1; j <= 1; j++) {
        for (int i = -1; i <= 1; i++) {
            vec2 g = vec2(float(i), float(j));
            vec2 o = hash22(n + g);
            vec2 delta = g + o - f;
            float d = dot(delta, delta);

            if (d < f1) {
                f2 = f1;
                f1 = d;
            } else if (d < f2) {

```

```

        f2 = d;
    }
}
return vec2(sqrt(f1), sqrt(f2));
}

```

5. Domain Warping & Fractal Synthesis

5.1 Fractal Brownian Motion (fBM)

Superimposing successive octaves of noise:

$f_{\text{fBM}}(\mathbf{x}) = \sum_{k=0}^{N-1} A^k \cdot \left(\Lambda^k \mathbf{x} \right)$

where A is the gain/persistence (< 1.0) and Λ is the lacunarity (> 1.0).

```

float fbm(in vec2 p, in int octaves, in float lacunarity, in float
gain) {
    float sum = 0.0;
    float amp = 1.0;
    float freq = 1.0;
    mat2 rot = rotate2D(0.5); // Decorrelates axes between octaves

    for (int i = 0; i < 8; i++) {
        if (i >= octaves) break;
        sum += amp * perlinNoise(p * freq);
        p = rot * p;
        freq *= lacunarity;
        amp *= gain;
    }
    return sum;
}

```

5.2 Multi-Scale Domain Warping

Feed the noise field back into its own coordinate vector to generate chaotic, fluid-like topological maps.

```

// Domain Warping: p' = p + \alpha * fbm(p + \beta * fbm(p))
float domainWarp(in vec2 p, out vec2 q, out vec2 r, in float time) {
    q = vec2(
        fbm(p + vec2(0.0, 0.0) + 0.05 * time, 5, 2.0, 0.5),
        fbm(p + vec2(5.2, 1.3) + 0.07 * time, 5, 2.0, 0.5)
    );

    r = vec2(
        fbm(p + 4.0 * q + vec2(1.7, 9.2) + 0.15 * time, 5, 2.0, 0.5),
        fbm(p + 4.0 * q + vec2(8.3, 2.8) + 0.126 * time, 5, 2.0, 0.5)
    );
}

```



```

    );

    return fbm(p + 4.0 * r, 6, 2.0, 0.5);
}

```

6. 2D Signed Distance Fields (SDFs) & Operators

A Signed Distance Field maps position p to the shortest scalar distance to a boundary: negative inside, positive outside, zero on the interface.

6.1 Primitives

```

// Circle: radius r
float sdCircle(in vec2 p, in float r) {
    return length(p) - r;
}

// Axis-Aligned Box: half-bounds b
float sdBox(in vec2 p, in vec2 b) {
    vec2 d = abs(p) - b;
    return length(max(d, 0.0)) + min(max(d.x, d.y), 0.0);
}

// Oriented Line Segment from a to b with thickness
float sdSegment(in vec2 p, in vec2 a, in vec2 b) {
    vec2 pa = p - a;
    vec2 ba = b - a;
    float h = clamp(dot(pa, ba) / dot(ba, ba), 0.0, 1.0);
    return length(pa - ba * h);
}

// Regular Equilateral Triangle
float sdEquilateralTriangle(in vec2 p, in float r) {
    const float k = sqrt(3.0);
    p.x = abs(p.x) - r;
    p.y = p.y + r / k;
    if (p.x + k * p.y > 0.0) p = vec2(p.x - k * p.y, -k * p.x - p.y) /
2.0;
    p.x -= clamp(p.x, -2.0 * r, 0.0);
    return -length(p) * sign(p.y);
}

// Annulus / Ring / Donut
float sdRing(in vec2 p, in float r, in float strokeWidth) {
    return abs(length(p) - r) - strokeWidth * 0.5;
}

```

6.2 Constructive Solid Geometry (CSG) & Smooth Blends

```
// Standard Boolean Operators
float opUnion(in float d1, in float d2) { return min(d1, d2); }
float opIntersection(in float d1, in float d2) { return max(d1, d2); }
float opSubtraction(in float d1, in float d2) { return max(-d1, d2); }

// Smooth Polynomial Minimum (smooth union)
float opSmoothUnion(in float d1, in float d2, in float k) {
    float h = clamp(0.5 + 0.5 * (d2 - d1) / k, 0.0, 1.0);
    return mix(d2, d1, h) - k * h * (1.0 - h);
}

// Smooth Polynomial Subtraction
float opSmoothSubtraction(in float d1, in float d2, in float k) {
    float h = clamp(0.5 - 0.5 * (d2 + d1) / k, 0.0, 1.0);
    return mix(d2, -d1, h) + k * h * (1.0 - h);
}

// Smooth Polynomial Intersection
float opSmoothIntersection(in float d1, in float d2, in float k) {
    float h = clamp(0.5 - 0.5 * (d2 - d1) / k, 0.0, 1.0);
    return mix(d2, d1, h) + k * h * (1.0 - h);
}
```

6.3 SDF Utility: Anti-Aliased Rendering

```
// Renders antialiased crisp edge for any 2D SDF
float renderSDF(in float d) {
    float antiAliasUnits = fwidth(d); // Matches screen-space
    derivative footprint
    return 1.0 - smoothstep(-antiAliasUnits, antiAliasUnits, d);
}

// Stroke / Outline extraction
float renderStroke(in float d, in float width) {
    float antiAliasUnits = fwidth(d);
    return 1.0 - smoothstep(width - antiAliasUnits, width +
antiAliasUnits, abs(d));
}
```

7. Procedural Tiling, Lattices & Symmetry

7.1 Domain Repetition

```
// Infinite repetition on periodic grid of size c
vec2 opRepetition(in vec2 p, in vec2 c) {
    return mod(p + 0.5 * c, c) - 0.5 * c;
}

// Clamped / Finite repetition within bounds
vec2 opRepetitionLimited(in vec2 p, in float c, in vec2 limit) {
    return p - c * clamp(round(p / c), -limit, limit);
}
```

7.2 Hexagonal Grid Tiling

Transforms Cartesian coordinates into a discrete hexagonal coordinate index and centered local UV.

```
struct HexGrid {
    vec2 id;          // Unique discrete cell index
    vec2 uv;          // Centered coordinates within local hexagon
};

HexGrid getHexGrid(in vec2 p) {
    vec2 r = vec2(1.0, 1.7320508); // (1.0, sqrt(3.0))
    vec2 h = r * 0.5;

    vec2 a = mod(p, r) - h;
    vec2 b = mod(p - h, r) - h;

    vec2 gv = dot(a, a) < dot(b, b) ? a : b;
    vec2 id = p - gv;

    return HexGrid(id, gv);
}
```

7.3 Truchet Labyrinth Tiling

```
vec2 truchetTile(in vec2 p) {
    vec2 id = floor(p);
    vec2 f = fract(p);

    // Random diagonal flipping based on cell index hash
    if (hash21(id) > 0.5) {
        f.x = 1.0 - f.x;
    }
}
```

```

    // Distance to circular arcs at opposite corners
    float d1 = length(f) - 0.5;
    float d2 = length(f - vec2(1.0)) - 0.5;

    return vec2(d1, d2);
}

```

8. Color Science & Spectral Synthesis

8.1 Inigo Quilez Cosine Palettes

Generates cyclic, harmonious, and highly optimized palettes using 4 configuration vectors:

$\mathbf{C}(t) = \mathbf{a} + \mathbf{b} \cos(2\pi(\mathbf{c}t + \mathbf{d}))$

```

vec3 cosinePalette(in float t, in vec3 a, in vec3 b, in vec3 c, in
vec3 d) {
    const float TAU = 6.28318530718;
    return a + b * cos(TAU * (c * t + d));
}

```

```

// Preset: Neon Twilight
vec3 paletteTwilight(in float t) {
    return cosinePalette(t,
        vec3(0.5, 0.5, 0.5),
        vec3(0.5, 0.5, 0.5),
        vec3(1.0, 1.0, 1.0),
        vec3(0.0, 0.33, 0.67)
    );
}

```

```

// Preset: Thermal Radiation / Magma
vec3 paletteThermal(in float t) {
    return cosinePalette(t,
        vec3(0.5, 0.5, 0.5),
        vec3(0.5, 0.5, 0.5),
        vec3(2.0, 1.0, 0.0),
        vec3(0.5, 0.20, 0.25)
    );
}

```

8.2 HSV to Linear sRGB

```

vec3 hsv2rgb(in vec3 c) {
    vec4 K = vec4(1.0, 2.0 / 3.0, 1.0 / 3.0, 3.0);
    vec3 p = abs(fract(c.xxx + K.xyz) * 6.0 - K.www);
    return c.z * mix(K.xxx, clamp(p - K.xxx, 0.0, 1.0), c.y);
}

```

```
}
```

8.3 Oklab Perceptual Uniform Color Space

Avoids non-linear hue shifts and chroma-dimming present in naive RGB interpolation.

```
// Linear sRGB to Oklab
vec3 rgbToOklab(in vec3 c) {
    float l = 0.4122214708 * c.r + 0.5363325363 * c.g + 0.0514459929 *
c.b;
    float m = 0.2119034982 * c.r + 0.6806995451 * c.g + 0.1073969566 *
c.b;
    float s = 0.0883024619 * c.r + 0.2817188376 * c.g + 0.6299787005 *
c.b;

    float l_ = pow(l, 1.0 / 3.0);
    float m_ = pow(m, 1.0 / 3.0);
    float s_ = pow(s, 1.0 / 3.0);

    return vec3(
        0.2104542553 * l_ + 0.7936177850 * m_ - 0.0040720468 * s_,
        1.9779984951 * l_ - 2.4285922050 * m_ + 0.4505937099 * s_,
        0.0259040371 * l_ + 0.7827717662 * m_ - 0.8086757660 * s_
    );
}

// Oklab to Linear sRGB
vec3 oklabToRgb(in vec3 c) {
    float l_ = c.x + 0.3963377774 * c.y + 0.2158037573 * c.z;
    float m_ = c.x - 0.1055613458 * c.y - 0.0638541728 * c.z;
    float s_ = c.x - 0.0894841775 * c.y - 1.2914855480 * c.z;

    float l = l_ * l_ * l_;
    float m = m_ * m_ * m_;
    float s = s_ * s_ * s_;

    return vec3(
        +4.0767416621 * l - 3.3077115913 * m + 0.2309699292 * s,
        -1.2684380046 * l + 2.6097574011 * m - 0.3413193965 * s,
        -0.0041960863 * l - 0.7034186147 * m + 1.7076147010 * s
    );
}

// Perceptually uniform color mixing
vec3 oklabMix(in vec3 colA, in vec3 colB, in float t) {
    vec3 labA = rgbToOklab(colA);
    vec3 labB = rgbToOklab(colB);
    return oklabToRgb(mix(labA, labB, t));
}
```

```
}
```

9. Analytical Lighting, Normals & Shading

9.1 Normal Reconstruction via Screen Derivatives & Finite Differences

Compute accurate normal vectors on 2D heightfields without analytical derivatives.

```
// Reconstruction from scalar height function H(uv)
// Central difference method for 3D surface mapping
vec3 reconstructNormal(in vec2 p, in float eps, in float strength) {
    // Replace fbm with your height map function
    float hC = fbm(p, 4, 2.0, 0.5);
    float hR = fbm(p + vec2(eps, 0.0), 4, 2.0, 0.5);
    float hT = fbm(p + vec2(0.0, eps), 4, 2.0, 0.5);

    vec3 va = vec3(eps, 0.0, (hR - hC) * strength);
    vec3 vb = vec3(0.0, eps, (hT - hC) * strength);

    return normalize(cross(va, vb));
}
```

9.2 Blinn-Phong & Schlick-Fresnel Illuminator

```
struct SurfaceMaterial {
    vec3 baseColor;
    float roughness;
    float specularPower;
    float f0; // Reflectance at normal incidence (typically 0.04 for
dielectrics)
};
```

```
vec3 evaluateBlinnPhong(
    in vec3 normal,
    in vec3 viewDir,
    in vec3 lightDir,
    in vec3 lightColor,
    in SurfaceMaterial mat
) {
    vec3 N = normalize(normal);
    vec3 V = normalize(viewDir);
    vec3 L = normalize(lightDir);
    vec3 H = normalize(L + V);

    // Diffuse Lambertian
    float NdotL = max(dot(N, L), 0.0);
```

```

    vec3 diffuse = mat.baseColor * lightColor * NdotL;

    // Specular Blinn-Phong
    float NdotH = max(dot(N, H), 0.0);
    float specFactor = pow(NdotH, mat.specularPower);

    // Schlick Fresnel approximation
    float VdotH = max(dot(V, H), 0.0);
    float fresnel = mat.f0 + (1.0 - mat.f0) * pow(1.0 - VdotH, 5.0);

    vec3 specular = lightColor * specFactor * fresnel;

    return diffuse + specular;
}

```

9.3 Fast Screen-Space Rim & Subsurface Approximations

```

// Rim / Backlit Glow: Enhances silhouette borders
float calculateRim(in vec3 normal, in vec3 viewDir, in float rimPower)
{
    float f = 1.0 - max(dot(normalize(normal), normalize(viewDir)),
0.0);
    return pow(clamp(f, 0.0, 1.0), rimPower);
}

// Approximate Thin Subsurface Scattering (Wrap Lighting)
float wrapLighting(in vec3 normal, in vec3 lightDir, in float wrap) {
    float NdotL = dot(normalize(normal), normalize(lightDir));
    return clamp((NdotL + wrap) / (1.0 + wrap), 0.0, 1.0);
}

```